ASSISTANT · 架构说明 v2

文件系统四模块：实现与交互关系

各自实现什么 · 彼此靠什么连 · 为什么必须这么连

M1 文件管理 · M2 传输与共享 · M3 安全 · M4 插件

一页结论

- 四个模块不是四个并列功能
- 是一条受控流水线 + 一个共享身份
- 跨模块只剩 4 个契约
- 其余耦合一律驳回

依据：架构评审修订后的方案 v2

OVERVIEW

一页看懂

职责怎么切

一个问题只归一个模块

- M1 看得见 · M2 出得去进得来
- M3 该不该、准不准、要不要保护
- M4 谁来加功能
- 落不进这四行的，先别当新模块

靠什么连

4 个契约 + 公开 API

- C1 FileRef · C2 事件
- C3 网关 · C4 插件入口
- 模块门面上批准过的公开 API
- 应用层编排作为组合根

边界在哪

网关只授权，执行器动手

- 只管经本应用发起的受控操作
- 完整性是「发现」，不是「阻止」
- capability 不是进程内沙箱
- 跨库不假设原子性，用恢复状态机

一句话结论：四个模块能各自长大，不靠约定俗成，而靠把耦合压到 4 个契约上。谁想加一条新的跨模块交互，先证明它落得进「七条边」之一 —— 落不进，就是职责重叠或契约漏了，回头改设计，而不是就地加接口。

本稿所有契约、边界与顺序均取自 docs/architecture/ 文件系统四模块 - 关联与方案 .md （提案 v2 ）

职责划分

四模块：各自回答一个问题

模块	回答的问题	主要输出	明确不回答
M1 文件管理	盘上有什么、在哪、怎么找到	索引快照 · 查询结果 · 分类 / 排除 · box 登记与去向	该不该被操作、怎么送出去
M2 传输与共享	文件怎么出去、怎么进来	传输作业（状态机 / 分块 / 校验 / 重试 / 恢复） · 配对 · 通道	文件是什么、能不能发
M3 安全	该不该允许、有没有被动过、怎么保护	判定 Allow/Ask/Deny · 完整性账本 · 审计 · 保险箱	文件在哪、怎么发
M4 插件	谁来加新能力、加的能力能碰什么	扩展点实现 · 能力声明 · 插件生命周期	以上任何一件事本身

验收式判据：任何新需求先往这四行里放。放不进去，说明它是第五个模块 —— 停下来讨论，而不是塞进现有模块。

M3 内部还要再分三档（发现 / 判定 / 保护），三者的验收方式完全不同 —— 见第 14 页

结构

四层结构：依赖只许向下

接入层 QML 搜索浮窗 · 主面板 · 托盘 · 插件（只经 IHostApi，不碰模块）

▼

模块层 M1 FileIndex · M2 TransferManager · M3 SecurityCenter · M4 PluginManager

▼

契约层 C1 FileRef · C2 FileSystemEvent · C3 FileOpGateway · C4 IHostApi + 能力

▼

基座层 FileIdentity · Snapshot · 事件分发器 · 各库访问组件 · Migrator · Config · Logger

三条组织规则

1. 可以调公开业务 API，不许依赖别人的内部实现头 / 私有表 / 私有协作者

2. 应用层是组合根：构造、注入、编排都在这里，别散到各处

3. 不许为消除每次调用而加临时抽象，也不许把协调逻辑收进一个新的全局门面

唯一的两处「向上」是接口倒置： IFilePolicy 由 M3 实现、 IHostApi 由应用层实现，基座只持指针

核心机制

受控操作：全应用只有一条路

① 受理

- 请求提交
- 调用主体校验
- 目标对象重新解析

② 决策

- 策略判定 Allow/Ask/Deny
- Ask = 尚未授权
- 必要时等待审批

③ 收口

- 执行前再次验证
- 执行 / 创建异步作业
- 记录结果 → 更新状态

网关的职责是「授权 + 协调」，不是「替所有人做 I/O 」：实际文件操作由受控执行组件完成，传输作业交给 M2 。
不要求每读一个数据块都重跑一次策略求值，但必须在校验边界检查源文件身份、版本与授权有效性。

失败关闭

策略异常 / 超时 / 无效结果 / 审批失效，一律 Deny ；长作业遇到规则更新或信任撤销，按作业状态机处理

覆盖范围

只管经本应用发起的受控操作；资源管理器、别的程序直接动手，网关拦不住，也不承诺拦

双源去重

网关返回结果与文件系统通知是两路信息，必须用操作 id 关联，不能当成两次独立变更

这条流水线同时是插件的越权防线：插件没有文件系统权限，落盘动作都变成带 actor 的受控请求

契约 ①

C1 FileRef：轻量值句柄，不是授权凭证

```
struct FileRef {
  quint64 id      = 0;   //  不透明句柄
  QString path;          //  可失效的定位提示
  qint64  size, mtime;   //  变化提示
  quint32 catMask;       // M1  派生
  quint8  flags;
  quint8  secLevel;      //  仅展示 / 粗筛
```

发送不变量：发送前校验源文件身份 · 传输中检测源文件变化 · 接收端分块校验 + 最终内容哈希

不是身份凭证

· 进程内不透明句柄 ≠ 物理身份（卷序列号 + FileId ）
· 路径是定位提示，不是稳定主键；文件可在两次调用之间被替换

不是快照保证

· size / mtime 只是快速变化提示，不是内容完整性证明
· id 必须定义进程重启、文件删除、句柄失效时的行为

不是授权凭证

· secLevel 只能展示与粗筛，授权时必须重新判定
· 敏感操作尽可能用已打开的句柄验证实际对象，避免 TOCTOU

字段可以随实现调整；下面三条语义不可省略。

要求「发出去的内容与授权时严格一致」时，必须用可验证快照或受控句柄 —— 只看 size/mtime 不够

契约 ②

C2 事件：变化通知，不是审计日志

它只做一件事

告诉订阅者「盘上变了」，用于增量处理。
它可能丢失、重复、乱序、溢出；它不能当审计日志，也不能当「没收到就等于没变」的依据。

协议必须先写死这 5 件事

1. seq 的作用域、生成方、比较规则（全局单调还是按根单调？）
2. Added / Removed / Renamed 的路径语义（目录重命名时子树怎么表达）
3. 可获得的物理身份信息，以及拿不到时怎么办
4. Overflow 对应哪个监控根 / 受影响范围
5. 重复事件、乱序事件、消费者失败的处理方式

消费者	收到 Overflow / 中断 / 失败时
M1 索引	重扫受影响范围，修复索引
M3 账本	该范围标「待复核」——不许因没收到后续事件就认定没变
M2 作业	对源 / 目标文件主动校验，不靠通知决定作业状态
其他	必须写出自己的恢复 / 重试 / 降级策略，写不出不许接入

消费者由应用层显式组装；固定数量只是当前约定，不是长期不变量 —— 要加就走显式依赖，不许开隐式全局订阅

契约 ③

C3 网关：授什么权、谁动手

组件	负责	不负责
FileOpGateway	请求校验 · 主体识别 · 策略调用 · 审批协调 · 执行前验证 · 结果关联	底层文件 I/O 、传输
IFilePolicy （M3）	同步、纯内存的规则求值	审批、落库、哈希、传输准备
受控执行组件	执行受控的本地文件操作	策略判定
TransferManager （M2）	异步传输作业与通道	绕过授权边界
审计组件	按风险分级持久化请求 / 判定 / 结果	阻塞前台

失败关闭（比 v1 更完整）

- 策略异常 / 超时 / 无效结果 → Deny
- 审批失效、规则更新、信任撤销 → 按作业状态机处理
- 用户取消 → 后续块级操作立即失效

静态检查的真实语义

- 覆盖 Qt / Win32 / Shell / 标准库全部变更 API
- 放行受控执行组件，拒绝 QML 层、插件、其他模块私自调用
- 验收不是「没命中字符串」，而是「所有路径都过边界」；例外逐条登记

网关实现为「注入依赖的具体协调类」；未来确有多种实现再抽接口 —— 不因形式统一而加一层空虚接口

契约 ④

C4 插件入口：能力声明，不是沙箱

```
class IHostApi {
  search(text, limit);          // index.read
  resolve(id);                  // file.read
  submitIngest(path, src);      // ingest.write
  requestSend(id, device);      // transfer.send → 必过授权
  showPanel(qmlUrl);            // ui.panel
};
```

必须说清的一句话

进程内插件能直接调 Win32 、读进程内存、影响宿主。所以 manifest 、签名、 capability 都只是「契约」，不是「边界」。

签名只证明来源与完整性，不证明行为可信 —— 这一点必须写在安装提示里，而不是靠用户自己发现。

v1 能力清单（8 个，名称集中定义，不许各处拼写）

index.read　file.read　file.write　op.execute

宿主必须自己做的两件事

- 由宿主解析实际调用主体，插件自填 actorId 一律无效
- 每次宿主 API 调用都校验能力（默认拒绝）
- API 版本兼容与能力授权分开判断（两个错误码，不许混）
- 生命周期：加载位置可信 → 校验 manifest → 初始化 → 失败干净回滚

依赖规则

合法的三条路，和非法的三条路

合法：三者之一

1. **契约类型**
   include/contract/ 里的 C1–C4
2. **公开业务 API**
   模块门面上明确批准过的方法，如 FileIndex::upsert()
3. **应用层编排**
   组合根串起来的调用顺序，不落在任何模块内部

驳回：典型的三种

1. **include 别人的内部实现头**
   #include "core/TransferManager.h" 出现在 M3 里
2. **直接查别人的私有表**
   M3 去读 M1 的 files 表 —— 表结构一改就崩
3. **绕过授权边界**
   M4 直调 M2 内部通道，等于自己给自己授权

契约数量：是审查提示，不是正确性指标

v1 写的是「只能有四个，多一个都不加」。 v2 改成：四个是当前目标集合，新增必须说得清「独立职责、必要性、依赖方向、失败语义」四件事。说得清就不必因为超过四个而绕道；说不清，一个也不该加。机械冻结数量只会逼出畸形绕过。

判据可以背下来：依赖的是契约类型、门面上的批准方法、还是应用层编排？三者之外一律驳回

交互关系

七条边：加交互前先落到某一条上

边	传递什么	走什么	失败模式与对策
M1 → M2	待发送的 FileRef 集合	TransferManager::enqueue()	排队期间文件被改 → 重新解析验身份，不匹配就跳过，绝不发半截
M2 → M1	新落地文件 + 来源信息	FileIndex::upsert() / remove()	索引未就绪 → 排队不丢；恢复后补提交，不重收文件
M1 → M3	查询视野 + 变更事件	query() + 订阅 C2	Overflow → 标待复核，巡检兜底； M3 不许自己扫盘
M3 → M1	密级回写 secLevel	快照重建（同 catMask ）	rulesVersion 驱动重建；授权时仍必须重新判定
M3 → M2	OpDecision + obligations	网关判定 → 创建受控作业	判定异常 / 审批失效 → Deny ； 执行中规则变更按作业状态机
M2 → M3	外来文件校验请求	SecurityCenter::verifyIncoming()	校验不符 → 留隔离区、不入库、不进索引
M4 → 全部	能力调用	IHostApi → 公开 API / 网关	能力不足给明确错误码；插件异常见第 15 页

七条边之外的新交互 = 新增一条边，需要重新评审：多半是职责重叠，或某个契约漏了一句话

M1 实现

文件管理：只补接缝，不重写

已有主体不动： FileIndex 门面 / FileSnapshot / SnapshotManager / A3–A8 清单 / 治理 B–E 。 M1 不承担 M2 的传输执行，也不承担 M3 的安全授权。

候选字段	加的条件	必须同时明确的语义
sec_level	作为 M3 密级结果的派生缓存，且能识别规则版本过期	授权时必须重新判定；只用于展示 / 筛选 / 粗筛，不能作为授权依据
prov_id	来源关联、查询性能或现有数据模型确有需要时，才进索引库	来源记录的权威存储位置、关联失效时的行为、删除与迁移策略

v1 → v2 的关键收紧：这两个字段不再「顺手一起加」

薄门面

M2 / M3 是客户，不是成员：只经 upsert / remove / query 通信

跨库不假设外键

来源记录在独立库时，用稳定标识 + 可恢复应用逻辑关联，不靠跨库外键

字段要有出处

新增字段必须服务明确的查询或恢复需求，不为「将来可能需要」无限扩

secLevel 若进快照： uint8 向量， 20 万条约 0.2 MB ，按 rulesVersion 重建，不重扫盘

M2 实现

传输：一套作业内核，三个交付单元

① 传输作业内核

状态机 · 分块 · 校验 · 重试 · 取消 · 恢复

先用本地模拟通道验证

不碰网络就能验收

② 渠道目录通道

复用 box / 追踪 / 归档机制

投递到渠道客户端监听的目录

验收入库与索引一致性

③ LAN 通道

配对 · 验证过的 TLS · 接收隔离区

分块续传 · 最终提交 · 崩溃恢复

①② 稳定后才做

接收端 6 条硬要求（都是安全用例，不是可选项）

路径： 远端不能决定最终落盘路径；拒绝穿越、 ADS 、保留设备名、危险 reparse point

入库： 未完成校验的文件不得进入正式索引；默认不覆盖同名文件

恢复： 接收 / 验证 / 入库 / 索引更新必须能在崩溃后恢复

状态： 源文件变化、重复块、任务取消、提交失败都要有明确状态转移

TLS 红线（v1 已修订）

目标环境无法提供「经过验证」的 TLS 后端 → 暂停 LAN 发布。

不允许自动退化成自研加密协议。应用层 AES-GCM 不是 TLS 的替代品 —— 两者解决的是不同层面的问题。

→ 这是一道前置门槛，不是可以留到发布后再补的项。

mDNS 不进首版：手动地址 + 配对码起步；后续按真实发现体验与依赖成本再评估，但不永久禁止

M3 实现

安全：发现 / 判定 / 保护，三种能力分开验收

发现

- 订阅事件 + 快速签名 + 周期复核
- 只承诺「在监控范围内发现变化」
- 不承诺阻止所有外部修改

判定

- 对经本应用的受控操作求值
- 返回 Allow / Ask / Deny
- 异常与超时一律失败关闭

保护

- 保险箱 + 密钥管理
- 受控的临时明文生命周期
- 只承诺静止态加密

账本：状态必须能说出「不知道」

正常　待复核　已变化　不可验证

监控中断、事件溢出、文件读取失败 → 显式标记，不许静默当成正常。

快速签名只做变化筛查；要判「内容是否一致」必须全量哈希，或分块哈希 + 最终哈希。

保险箱 v1 必须明确的 5 件事

1. 数据密钥的生成与管理（AES-256-GCM + DPAPI 保护）
2. nonce 不重复的保证（随机碰撞论证或计数器方案）
3. 文件名 / 元数据与密文之间的认证关系（AAD 绑什么）
4. 恢复码的生成、保存、验证与密钥恢复语义
5. 临时明文的创建 / 清理 / 异常恢复；改文件后如何重加密

擦除的诚实边界： 删掉临时文件不等于 SSD / 日志 / 备份 / 分页文件里的明文副本都消失 —— 只承诺「受控的临时明文生命周期」，不承诺无法验证的绝对擦除。

插件：进程内完全信任，先把边界说清

v1 的信任模型

面向「用户主动安装并明确授权」的可信代码，进程内加载。

它能直接调 Win32 、读进程内存、影响宿主 —— 签名只用来识别来源与完整性，不用来证明行为可信。

承认：插件崩溃可能拖垮主进程。这是 v1 的已知代价，不是缺陷遗漏。

宿主必须做的 6 件事

1. 从可信加载位置加载，明确安装与启用流程
2. 校验 manifest 、 API 版本、能力声明
3. 由宿主识别插件身份，不信任自填 actorId
4. 每次宿主 API 调用都校验能力（默认拒绝）
5. 管理停用 / 卸载 / 初始化失败 / 宿主关闭
6. 异常给出明确错误信息，不静默吞掉

两类扩展点之外，还要三个稳定面

ISearchProvider
往搜索结果里加一类结果

IFileActionProvider
给 FileRef 加动作（右键 / 命令）

IIngestSource
提供新的文件来源

版本与能力分开判断： apiVersion 不兼容 → 拒绝加载；能力不足 → 拒绝调用。两个错误码不许混，否则用户只会看到「插件不工作」。
进程外插件与 IPC 隔离属于后续候选，与 mDNS 是两件独立的事，不互相绑定。

5 个存储区：谁是权威，失败怎么办

存储区	性质	唯一写者	迁移失败
file_index.db	派生（可从磁盘重建）	FileIndexDatabase	备份后重建，代价是重扫
downloads.db	权威（ box / 去向链重建不出来）	DownloadsDatabase	保留数据，停止相关功能，拒绝启动
security.db	权威（账本 / 审计 / 密钥清单）	SecurityDatabase	同上，且比 downloads 更硬
transfer.db	半权威（作业可弃，配对信任不可弃）	TransferDatabase	降级启动：禁传输，提示重新配对
plugins/<id>/	插件私有（命名空间隔离）	插件自己	隔离该插件，不破坏主程序

跨库状态机的四问（每个状态都要答）

1. 哪些持久化操作已经完成?
2. 哪些可以安全重试?
3. 哪些必须补偿或回滚?
4. 哪些只能交给用户处理?

两个必须写成用例的场景

- 已写入 quarantine 但未入库 → 重启后续验，或安全清理，不许停在半入库
- 来源登记成功但索引更新失败 → 补提交索引，不重收整个文件

独立数据库 ≠ 天然一致：跨库流程必须有恢复入口和验收用例，这是 v1 只写「靠应用逻辑维护」补不上的

落地

依赖阶梯：能并行的并行，不能被同一闭环卡住的先卡死

现在	可并行	本地闭环	传输三级
内核 A3–A6 身份 · 事件恢复 · 受控操作边界	**B 治理过滤** 分类 · 排除 · 查询语法	**C/D box + 追踪** 入库恢复闭环	**M2① 作业内核** 本地模拟通道
	M3-p1 安全 边界 + 判定 + 账本骨架	**E 归档多源** 不被 LAN 待定阻塞	**M2② 渠道通道** 复用 box / 追踪
			M2③ LAN 通道 TLS 实测通过才做

旁支： M3-p2 保险箱

独立验证加密 / 密钥 / 恢复码 / 临时明文；可复用验证过的加密工具，但不依赖 M2 业务模块，因此不必等传输。

条件触发： M4 插件

挂在「核心公开 API 定型」这个条件上，不挂在某条实现链的末尾 —— 所以它不看传输做没做完，看接口稳没稳。

并行与串行的界线：互不依赖的实现可以并行；被同一闭环依赖的接口必须先各自验收

收口

已拍板的、还没测的、明确不做的

四项决策（已定）

操作边界
做，但网关只授权协调

mDNS
首版不做，也不永久禁止

插件信任
进程内完全信任，无沙箱

保险箱
只承诺静止态加密

待实测（不是设计待决）

Qt TLS 后端
LAN 发布的前置门槛

prov_id 是否入索引
看查询需求与性能实测

mDNS 是否引入
看真实发现体验与成本

插件进程外化
与 mDNS 无关，独立决策

v1 明确不做

- 文件系统过滤驱动级拦截
- 全系统外部操作强制控制
- 进程内插件沙箱
- 运行期完全无明文的保险箱
- 自研 TLS 替代协议
- NAT 穿透 / 公网直连

这些不是「以后再说」，是「说了也做不到」——写进文档是为了不给自己留假承诺的余地。

所有性能数字都是目标值（待实测），不是已实现能力的声明

小结

四个模块的关系，是「一段流水线 + 一个身份 + 少数契约」

M1 管看得见， M2 管出得去进得来， M3 管该不该 / 准不准 / 要不要保护， M4 管谁来加功能。

四者共用一个身份（ FileRef / FileIdentity ）、一条受控流水线、一小组契约与批准过的公开 API 。

其余一切耦合都是设计失误 —— 包括「就这一次，先直接调一下」。

下一步：落 M3-p1 的契约头 + 网关骨架 + 边界检查